

Towards Long-horizon Embodied Agents with Tool-Aligned Vision-Language-Action Models

Zixing Lei^{1,2*} Changxing Liu^{1*} Yichen Xiong¹ Minhao Xiong¹ Yuanzhuo Ding¹
 Zhipeng Zhang¹ Weixin Li^{2,3} Siheng Chen^{1†}

¹Shanghai Jiao Tong University ²Zhongguancun Academy

³Beihang University

chezacarss@sjtu.edu.cn cx-liu@sjtu.edu.cn sihengc@sjtu.edu.cn

Abstract

Vision-language-action (VLA) models are effective robot action executors, but they remain limited on long-horizon tasks due to the dual burden of extended closed-loop planning and diverse physical operations. We therefore propose **VLAs-as-Tools**, a strategy that distributes this burden across a high-level vision language model (VLM) agent for temporal reasoning and a family of specialized VLA tools for diverse local physical operations. The VLM handles scene analysis, global planning, and recovery, while each VLA tool executes a bounded subtask. To tightly couple agent planning with VLA tool execution in long-horizon tasks, we introduce a *VLA tool-family interface* that exposes explicit tool selection and in-execution progress feedback, enabling efficient event-triggered agent replanning without continuous agent polling. To obtain diverse specialized VLA tools that faithfully follow agent invocations, we further propose *Tool-Aligned Post-Training* (TAPT), which constructs invocation-aligned training units for instruction following and adopts tool-family residual adapters for efficient tool specialization. Experiments show that VLAs-as-Tools improves the success rate of $\pi_{0.5}$ by 4.8 points on LIBERO-Long and 23.1 points on RoboTwin, and further enhances invocation fidelity by 15.0 points as measured by Non-biased Rate. Code will be released.

1 Introduction

Recent vision-language-action (VLA) models, including OpenVLA-OFT, RDT-1B, and GR00T N1, have advanced general-purpose robot control and cross-embodiment generalization [Kim et al., 2025, Liu et al., 2024, NVIDIA et al., 2025]. Yet they are often used as end-to-end embodied controllers, requiring a single policy to interpret goals, perceive scenes, predict actions, and coordinate multi-step execution. This setting is difficult for current VLAs: long-horizon tasks require decomposition, progress tracking, failure recovery, and skill composition, while embodied data remains limited and model scale still constrains planning. As a result, VLAs provide powerful language-conditioned robot-control capabilities, but remain limited as standalone long-horizon agents.

There are two routes toward long-horizon embodied tasks. One direction keeps planning inside end-to-end VLA policies, where models such as $\pi_{0.5}$ introduce semantic subtask prediction and task-level planning tokens within the VLA backbone [Physical Intelligence et al., 2025]. Another direction, represented by SayCan and Code as Policies, adopts an agentic formulation in which language models select skills, compose programs, or invoke robot APIs [Ahn et al., 2022, Liang et al., 2022]. This follows the successful paradigm of digital agents, where high-capacity LLMs handle goal decomposition, state tracking, replanning, and error recovery, while tools provide bounded,

*Equal contribution

†Corresponding author

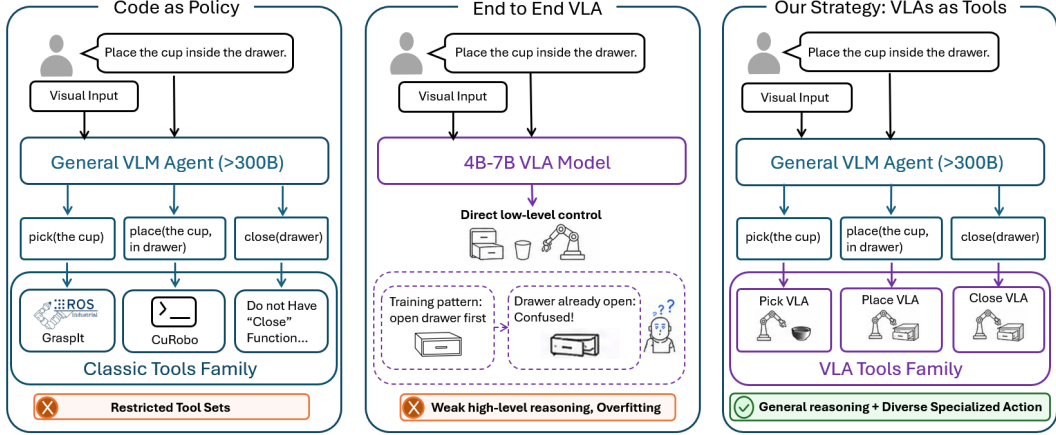


Figure 1: Motivation and overview of *VLAs-as-Tools*. Classic code-as-policy systems benefit from high-level reasoning, but are limited by manually specified and restricted robot tool sets. End-to-end VLAs provide stronger robot-control capabilities, but make long-horizon execution vulnerable to demonstration biases and recovery failures. Our method combines the strengths of both paradigms: a general VLM agent decomposes the task and invokes a family of multiple VLA tools for execution, where each tool provides bounded, language-conditioned physical execution.

observable, and reliable execution interfaces [Yao et al., 2022, Schick et al., 2023, Yang et al., 2024]. However, existing embodied tools are often manually specified and task-specific, making them weaker physical execution than modern VLAs. This leaves embodied AI with a complementary gap: end-to-end VLAs provide stronger physical execution but weaker long-horizon reasoning, whereas existing embodied agents provide stronger planning but rely on weaker physical tools.

To bridge this gap, we propose *VLAs-as-Tools*, a strategy for solving long-horizon embodied tasks by enabling a planning-capable VLM agent to invoke bounded VLA executions as physical tools. This strategy addresses the limitations of individual VLAs by distributing the burden across time length and task breadth: the VLM extends execution over long horizons through planning, state tracking, and recovery, while multiple specialized VLA tools cover diverse bounded subtasks. Each VLA therefore operates within a focused execution scope, where it can be efficiently adapted from task-specific demonstrations and reliably map visual-language inputs to continuous robot actions. However, standalone VLAs are not readily tool-usable, since their execution can be biased by scene priors, demonstration regularities, or visual context rather than the invoked instruction [Zhang et al., 2026b, Fang et al., 2026], and task-specific adaptation can degrade pretrained semantic understanding and generalization [Hancock et al., 2025, Liu et al., 2026a]. Thus, realizing *VLAs-as-Tools* requires addressing two core challenges: how to make VLAs callable and reliable physical tools, and how to integrate them effectively into an embodied agentic system.

To make VLAs tool-usable, we introduce *Tool-Aligned Post-Training* (TAPT), a VLA-level adaptation method for producing a family of capable VLA tools with distinct bounded-subtask specialties. TAPT constructs subtask-centric data by pairing bounded subtasks with precise language instructions, so that each VLA tool learns a clear correspondence between the agent’s invocation and the intended physical behavior. It further adopts tool-family residual adapters to parameter-efficiently create many specialized VLA tools on top of a shared backbone, while isolating tool-family-specific adaptation and mitigating degradation of pretrained semantic generalization. As a result, TAPT turns a general VLA into a scalable set of strong, instruction-faithful embodied tools that can be selected and composed by the agent across diverse subtask calls.

To integrate VLAs effectively into an embodied agentic system, we contribute a *VLA tool-family interface* that tightly couples the low-level VLA tool family with the high-level planner. The interface organizes multiple specialized VLA tools into a single family, where each tool is adapted for a distinct bounded subtask. It defines two typed message sets: agent-to-tool invocation messages that specify bounded subtasks and route them to the appropriate VLA tool, and tool-to-agent feedback messages that return in-execution state signals. The former grounds planned subtask sequences in the correct physical executor, while the latter exposes timely intervention points for long-horizon execution, supporting low-frequency but responsive replanning.

We evaluate our method at both the policy level and the agent-loop level. Policy-level experiments measure whether TAPT improves instruction-aligned execution for bounded subtask calls, while agent-loop experiments assess the overall effectiveness and stability of the embodied agentic system in long-horizon control. Across OpenVLA, OpenVLA-OFT, and $\pi_{0.5}$ on LIBERO-Long, CALVIN, and RoboTwin, *VLAs-as-Tools* consistently improves VLA tool usability and embodied agentic execution. Notably, it increases the success rate of OpenVLA-OFT on RoboTwin by 35.5 points. It also improves invocation fidelity by 16.2 points in Non-biased Rate.

2 Related Work

Embodied Foundation Models Recent progress in embodied foundation models has been driven by Vision-Language-Action (VLA) policies, which directly map visual observations and language instructions to robot actions using large pretrained vision-language backbones Brohan et al. [2023b], Octo Model Team et al. [2024], Kim et al. [2024, 2025], Black et al. [2024], Physical Intelligence et al. [2025], NVIDIA et al. [2025]. Beyond short-horizon control, recent VLAs further introduce subtask prediction, hierarchical action generation, progress modeling, or post-training mechanisms to improve long-horizon and instruction-conditioned manipulation Yang et al. [2025a], Fan et al. [2025], Liu et al. [2026b], Sun et al. [2026], Wen et al. [2025], Hancock et al. [2025]. However, strong task-level performance does not necessarily imply that a VLA faithfully grounds each commanded behavior in language. Prior studies show that language-conditioned policies can exploit visual priors, dataset regularities, or common demonstration patterns instead of treating language as the primary control signal Jang et al. [2022], Mees et al. [2022], Glossop et al. [2025], Fei et al. [2025]; counterfactual evaluations further reveal cases where visual cues override the specified instruction Fang et al. [2026], and recent work studies how to restore or preserve linguistic grounding during VLA adaptation Zhang et al. [2026b], Hancock et al. [2025]. This issue becomes critical when a VLA is placed behind a high-level planner: the planner may repeatedly issue local subtask invocations, but a standalone VLA is not trained to preserve the semantics of each invocation under such agent-tool interaction. Our work shifts the focus from standalone VLA task success to invocation-aligned VLA execution, using subtask-centric data, invocation-aligned training, and tool-family residual adapters to strengthen the binding between the agent command, the selected tool family, and the resulting behavior.

Embodied Agentic Systems Recent embodied AI advances have increasingly focused on agentic systems, particularly in language-conditioned and long-horizon tasks Salimpour et al. [2025], Liang et al. [2025]. Hierarchical embodied systems commonly address such tasks by decomposing high-level goals into task-level decisions and executable low-level skills Ahn et al. [2022], Huang et al. [2022], Liang et al. [2022], Belkhale et al. [2024], Shi et al. [2025], NVIDIA et al. [2025], but they often rely on hand-engineered skills, affordance models, latent options, or jointly trained executors. More recent systems scale this paradigm with learned skills and agentic execution loops. Agentic Robot couples a reasoning model, a VLA executor, and a temporal verifier for closed-loop embodied execution Yang et al. [2025b]. ThinkAct connects high-level embodied reasoning and low-level execution through reinforced visual latent planning Huang et al. [2025]. AtomicVLA builds a planning-and-execution framework around atomic skill abstractions and skill-guided expert specialization Zhang et al. [2026a], while RoboClaw uses a VLM-driven controller to orchestrate learned policy primitives for long-horizon tasks and autonomous data collection Li et al. [2026]. LiLo-VLA further shows that modular object-centric policies can improve compositional long-horizon manipulation through dynamic replanning and skill reuse Yang et al. [2026]. These works validate the importance of skill decomposition and modular policy composition. Our distinction is the VLA-side execution interface itself: simply wrapping a standard VLA with a planner is insufficient, so the low-level executor must be post-trained around the same invocation unit used by the agent. We expose pretrained VLAs as reusable embodied tools whose invocations pair bounded language instructions with explicit tool-family selectors, and we make the selected tool family executable through tool-family residual adapters rather than treating it only as an additional language token.

3 Formalizing VLAs as Callable Tools

This section formalizes the *VLAs-as-tools* strategy: a VLA is not used as a monolithic long-horizon policy, but as a family of bounded, callable executors inside an embodied agentic control loop. The connection between the high-level agent and the VLA tools is an interface $\mathcal{I} = (\mathcal{C}, \mathcal{R})$, where $\mathcal{C}$ is the set of agent-to-tool invocation messages and $\mathcal{R}$ is the set of tool-to-agent feedback messages. A user specifies a goal g , and a high-level agent Π_ϕ maintains an agent-side state s_k over observations,

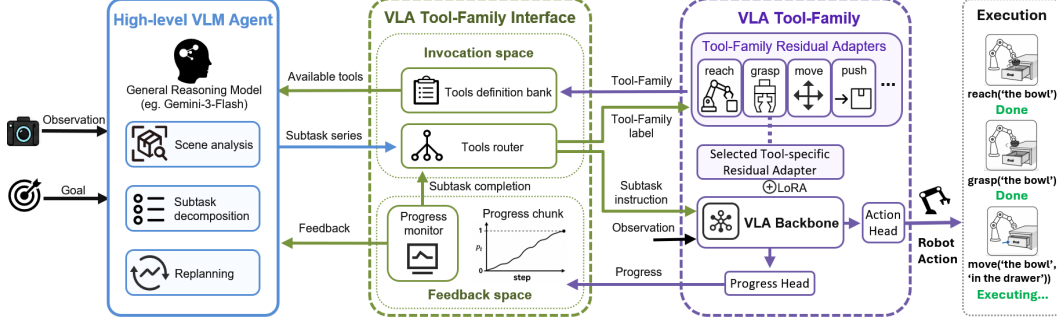


Figure 2: Overview of the closed-loop system with the VLAs-as-Tools strategy. A VLM agent performs high-level, long-horizon planning, while the VLA tool-family interface guides explicit tool calls and provides feedback for low-frequency agent intervention. VLAs become callable tools that faithfully execute invocations, with a parameter-efficient tool family generated via residual adapters.

previous calls, and returned feedback. At decision step k , the agent sends an invocation message through the interface,

$$c_k = \Pi_\phi(q, s_k), \quad c_k = (g_k, z_k) \in \mathcal{C} = \mathcal{G} \times \mathcal{Z}, \quad (1)$$

where g_k is a discrete tool-family label, such as *grasp*, *open*, or *place*, and z_k is a scene-grounded subtask instruction. The tool-family label g_k selects a member of the VLA tool family, $\mathcal{T} = \{T_g\}_{g \in \mathcal{G}}$, where each T_g is a callable VLA tool specialized for one tool family. The instruction z_k grounds this tool family in the current scene by specifying the object, relation, and desired local effect.

The selected tool T_{g_k} executes the call over a bounded low-level horizon. Given robot observations o_t , it produces actions and a bounded trajectory

$$a_t \sim T_{g_k}(\cdot \mid o_t, z_k, h_t), \quad \tau_k = (o_{t_k:t_k+H_k}, a_{t_k:t_k+H_k-1}), \quad (2)$$

where h_t denotes an optional short execution history and H_k is the call horizon. After or during execution, the selected tool returns feedback r_k such as progress or completion information. The agent state is then updated as

$$s_{k+1} = U(s_k, c_k, \tau_k, r_k), \quad r_k \in \mathcal{R}, \quad (3)$$

and the loop repeats until the task terminates. Thus, the interface defines both directions of communication: $\mathcal{C}$ tells the selected tool what to execute, and $\mathcal{R}$ tells the agent what happened during execution. The invocation message set $\mathcal{C} = \mathcal{G} \times \mathcal{Z}$ is bounded and agent-visible: g_k chooses the tool family, while z_k grounds the selected tool family into a concrete scene-level subtask. This formalization turns long-horizon robot control into repeated bounded tool invocations, where the key methodological question is how to construct a VLA tool family $\mathcal{T}$ whose members can be reliably invoked, monitored, and composed by the high-level agent.

4 Methods

We instantiate the formal strategy above with two concrete design choices. First, we make a VLA callable through a VLA tool-family interface: the agent selects a tool family, provides a scene-grounded local instruction, and invokes the VLA for a bounded execution window. The VLA tool is therefore not a standalone task policy, but a bounded executor that receives an agent-specified invocation and returns progress feedback. Second, we train the VLA to follow this interface through Tool-Aligned Post-Training (TAPT), which aligns the training data, adapter structure, and optimization objectives with the same invocation unit used at test time.

4.1 Bidirectional Agent–Tool Interaction via a VLA Tool-Family Interface

Section 3 formulates the agent–tool interface as $\mathcal{I} = (\mathcal{C}, \mathcal{R})$. We instantiate this interface for VLAs-as-tools through two typed message sets: $\mathcal{C}$ contains invocation messages sent by the high-level agent to a selected VLA tool, and $\mathcal{R}$ contains feedback messages returned by the tool for monitoring.

4.1.1 Invocation Messages $\mathcal{C}$

Each VLA tool call has two fields: a tool-family label and a scene-grounded instruction. The tool-family label $g_k \in \mathcal{G}$, such as *grasp*, *open*, or *place*, selects one callable member from the VLA tool

family $\mathcal{T} = \{T_g\}_{g \in \mathcal{G}}$. The instruction $z_k \in \mathcal{Z}$ then grounds the selected tool family in the current scene by specifying the object, relation, and desired local effect. Together, these two fields form the invocation message $c_k = (g_k, z_k)$, so $\mathcal{C} = \mathcal{G} \times \mathcal{Z}$.

This factorization makes the agent’s request inspectable. The tool-family label g_k selects the tool family, while z_k grounds that tool family in the current scene. This is useful because visually similar instructions may require different tool families: *grasp*, *open*, and *place* can involve overlapping objects but differ in action distributions and termination conditions. The explicit tool-family label also gives TAPT a stable signal for organizing data and parameters, around tool-family specialization.

4.1.2 Feedback Messages $\mathcal{R}$

Feedback messages specify what the selected VLA tool reports while it is executing an invocation. We use a continuous progress signal $p_t \in [0, 1]$ as the main feedback message, where values near 0 indicate little progress and values near 1 indicate that the current invocation is nearly complete. This signal is local to the bounded call $c_k = (g_k, z_k)$: it measures progress on the selected tool invocation, not success of the full long-horizon task. Progress feedback is useful because the agent needs timely information before a subtask fully succeeds or fails. Repeatedly querying a large VLM to inspect execution is costly, while a binary completion signal can delay recovery. In contrast, a tool-side progress signal exposes intermediate execution states, such as stagnation or deviation, without requiring the agent to reason over every low-level action. In the interface, temporal progress is stored in the progress chunk, while a threshold-based monitor determines whether to advance, replan, or continue the current invocation. The VLA predicts progress with an auxiliary head $\hat{p}_t = \psi_\omega(b_t)$ attached to the backbone feature b_t , with supervision described in Section 4.2.

4.2 Improving VLA Tool Usability with Tool-Aligned Post-Training

Building on the VLA tool-family interface in Section 4.1, this section describes how we train a base VLA to behave like a reliable callable tool. TAPT follows the same pipeline as the agent–tool loop. First, it rewrites both imitation and reinforcement-learning supervision around bounded invocations: demonstrations are segmented into invocation-labeled action windows, and RL rollouts are initialized and rewarded at the same subtask level. Second, it makes the tool-family label executable by routing each invocation through tool-family residual adapters on top of a shared VLA backbone. Third, it post-trains the resulting model to both execute the requested local behavior and predict progress feedback for that invocation. Standard downstream SFT and RL are then used as optimization settings for adapting and evaluating the same invocation-aligned tool family.

4.2.1 Invocation-Aligned Training Units

TAPT first rewrites imitation learning around the unit the agent calls at inference time. Instead of training only on full-task trajectories, we segment demonstrations into bounded windows and annotate each window with an invocation $c = (g, z)$, where $g \in \mathcal{G}$ identifies the tool family and $z \in \mathcal{Z}$ describes the local scene effect. Each imitation unit is $x_{\text{IL}} = (o_{1:T}, z, g, a_{1:T}, p_{1:T}^*)$, where $o_{1:T}$ and $a_{1:T}$ are the observations and actions inside the window, and $p_{1:T}^*$ is a progress target or proxy. Segments are obtained from task annotations, state changes, contact events, and automatic multimodal labeling; the labeler assigns g using operational definitions based on contact pattern, force direction, and object motion. Detailed in Appendix A.

The same invocation view is used for reinforcement learning. For each call (g, z) , we build a bounded subtask rollout rather than starting from the original full-task initial state. The rollout starts from states where that invocation should begin, obtained from demonstration boundaries or successful executions of preceding subtasks. It is evaluated with a local completion predicate $\psi_{z,g}(s)$, such as contact, containment, relative pose, joint displacement, or object-on-surface relations. The rollout receives reward 1 if this predicate becomes true within horizon H , and 0 otherwise; the episode ends when the predicate is satisfied or the horizon is reached.

Thus IL and RL share the same target: reliable execution of the current bounded invocation. IL teaches the actions and progress for a call, while RL tests whether the same call can be completed from states where the agent would invoke it.

4.2.2 Tool-Family Residual Parameterization

The tool-family label g should control the VLA’s execution path, not merely appear as an extra language token. Tool families such as grasping, placing, opening, and rotating may share the same scene context, but they require different action distributions, termination conditions, and progress

Table 1: Main results under imitation learning. We report success rate (SR) on each benchmark; red values in parentheses indicate the absolute improvement over the corresponding single-model SFT baseline with the same VLA backbone. TAPT denotes Tool-Aligned Post-Training.

Base Model	Adaptation	Deployment mode	LIBERO-Long SR $\uparrow$	RoboTwin SR $\uparrow$
OpenVLA	SFT	Standalone	77.2	1.9
	SFT	Direct planner	77.0	13.8
	TAPT	VLA tool-family interface	82.4 ($\uparrow 5.2$)	5.7 ($\uparrow 3.8$)
OpenVLA-OFT	SFT	Standalone	92.0	16.9
	SFT	Direct planner	80.2	18.8
	TAPT	VLA tool-family interface	95.6 ($\uparrow 3.6$)	52.4 ($\uparrow 35.5$)
$\pi_{0.5}$	SFT	Standalone	92.4	39.4
	SFT	Direct planner	80.4	30.0
	TAPT	VLA tool-family interface	97.2 ($\uparrow 4.8$)	62.5 ($\uparrow 23.1$)

semantics. We therefore implement the VLA tool family with a shared pretrained backbone and deterministic tool-family residual adapters selected by g .

Concretely, let f_Θ be a pretrained VLA backbone. For each selected linear layer W , we add a bank of low-rank residual adapters $\{\Delta W_g\}_{g \in \mathcal{G}}$, with one adapter per tool family:

$$\Delta W_g = B_g A_g, \quad r = \text{rank}(\Delta W_g) \ll \text{rank}(W). \quad (4)$$

Given hidden activation x and selected family g , the adapted layer computes $y = Wx + \Delta W_g x$. Thus g deterministically routes the call to a lightweight LoRA residual Hu et al. [2021].

At inference time, the agent provides $c_k = (g_k, z_k)$. The selected residual path remains active for the duration of the bounded invocation, and actions are produced as

$$a_t \sim \pi_{\theta, \phi_{g_k}}(\cdot \mid o_t, z_k, h_t), \quad (5)$$

where ϕ_{g_k} denotes the residual-adapter parameters for family g_k . The same backbone features are also used by the progress head defined in Section 4.1. The adapters give each tool family a distinct execution path while preserving shared visual-language representations. Because each adapter is low-rank, adding tool families introduces only a small parameter overhead.

4.2.3 Tool-Aligned Post-Training Objective

TAPT post-trains the parameterization above on a broad invocation-aligned corpus $\mathcal{D}_{\text{mid}}^{\text{inv}}$ before benchmark-specific adaptation. The goal is to establish reusable semantics for the interface call $c = (g, z)$: g selects an execution path, z grounds that path in the scene, and the progress target teaches the feedback signal returned to the agent.

For imitation learning, TAPT combines two supervised signals. The action-cloning term trains the selected residual path ϕ_g to reproduce the actions for the invocation, while the progress term trains the feedback head to predict invocation progress:

$$\mathcal{L}_{\text{TAPT}} = \mathbb{E}_{(o, z, g, a, p^*) \sim \mathcal{D}_{\text{mid}}^{\text{inv}}} \left[- \sum_{t=1}^T \log \pi_{\theta, \phi_g}(a_t \mid o_t, z, h_t) + \frac{\lambda_{\text{prog}}}{T} \sum_{t=1}^T \|\hat{p}_t - p_t^*\|_2^2 \right]. \quad (6)$$

This objective jointly updates the tool-family residual adapters and progress head, so the resulting VLA learns both sides of the interface: executing the invocation and returning progress feedback.

For reinforcement learning, TAPT keeps the same invocation unit and optimizes the bounded subtask reward defined above. The reward evaluates only whether the current invocation is completed, rather than whether the full long-horizon task succeeds. In experiments we instantiate this RL stage with GRPO Shao et al. [2024]; implementation details are provided in Appendix B. Downstream SFT and RL reuse the same interface and objectives, replacing $\mathcal{D}_{\text{mid}}^{\text{inv}}$ with benchmark-specific invocation-aligned data or environments. The result is a VLA tool family $\mathcal{T}_{\theta, \Phi} = \{T_g\}_{g \in \mathcal{G}}$ that accepts $c = (g, z)$, executes the selected tool, and returns progress feedback.

5 Experiment

5.1 Experimental Setup

Our experiments ask four questions. First, can the VLAs-as-tools strategy improve long-horizon embodied performance? Second, does Tool-Aligned Post-Training make VLA execution more faithful

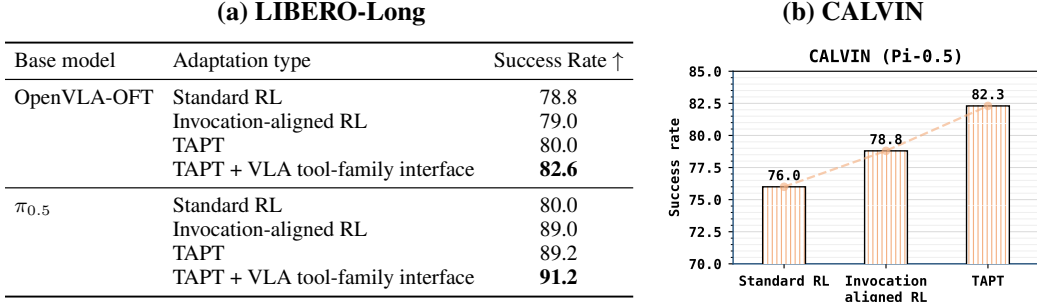


Figure 3: Main reinforcement-learning results on LIBERO-Long and CALVIN. LIBERO-Long evaluates standard RL, TAPT, and VLA tool-family interface deployment; CALVIN tests whether the gains persist under the benchmark’s native subtask structure rather than our split construction.

Table 2: Invocation fidelity on LIBERO-CF-Long benchmark under TAPT component ablations. Faithful Rate measures completion of the invoked counterfactual goal, while Non-biased Rate measures avoidance of the original LIBERO-10 source-task bias.

TAPT Components			OpenVLA-OFT		$\pi_{0.5}$	
IA SFT	IA Post-train	Residual adapters	Faithful (%) $\uparrow$	Non-biased (%) $\uparrow$	Faithful (%) $\uparrow$	Non-biased (%) $\uparrow$
			19.4	31.2	24.8	39.6
✓			30.4	38.8	36.6	43.2
✓	✓		40.6	39.2	38.2	46.6
✓	✓	✓	54.0	47.4	54.8	54.6

to agent invocations? Third, which components of the strategy are responsible for the gains? Fourth, does Tool-Aligned Post-Training improve downstream data efficiency in few-shot adaptation?

We evaluate our method mainly with two representative VLA backbones, OpenVLA-OFT and $\pi_{0.5}$, and include OpenVLA as an additional supervised-adaptation baseline when useful Kim et al. [2024, 2025], Physical Intelligence et al. [2025]. Experiments are conducted on LIBERO Liu et al. [2023], RoboTwin Mu et al. [2025], and CALVIN Mees et al. [2021]. For LIBERO, we use LIBERO-Long to highlight performance on long-horizon tasks. Since our data-splitting pipeline targets single-arm manipulation, RoboTwin evaluation uses 8 single-arm-executable tasks with a Franka embodiment; the task list and selection rule are provided in Appendix D. For CALVIN, we use CALVIN_D with an 80%/20% train-test episode split, treating it as a complementary testbed with existing subtask-level structure. TAPT includes two invocation-aligned (IA) stages: *IA Post-train*, an intermediate post-training stage on DROID-split, and *IA SFT*, downstream supervised adaptation on benchmark-specific datasets such as LIBERO-Long-split and RoboTwin-split. The “-split” suffix denotes demonstrations processed by our data-splitting pipeline into invocation-aligned subtasks. In the IL training of both OpenVLA-OFT and $\pi_{0.5}$, IA Post-train is conducted for 1 epoch; during IA SFT, LIBERO is trained for 150K/30K steps and RoboTwin for 60K/30K steps, respectively, with batch sizes of 8/256 following the official configurations. All RL experiments are implemented with RLinf, with implementation details provided in Appendix B. Success rate denotes task-level success with evaluation protocol details provided in Appendix D.

5.2 Can the VLAs-as-tools Strategy Improve Long-Horizon Embodied Performance?

Supervised downstream adaptation. We first evaluate imitation-learning adaptation when downstream demonstrations are available. Table 1 compares three deployment modes: a standalone SFT-trained VLA, a standard SFT-trained VLA with a direct planner, and our TAPT-trained VLA with the VLA tool-family interface. Deployment mode denotes how the trained VLA is used at evaluation time: *Standalone* executes the VLA as a single task policy, *Direct planner* lets a high-level planner periodically issue language instructions, and the *VLA tool-family interface* establishes a tightly coupled bidirectional interaction between the agent planner and VLA tools. The results show that simply placing a standard SFT policy inside an agent loop is insufficient: on LIBERO-Long, direct planner deployment changes performance by -0.2 , -11.8 , and -12.0 points for OpenVLA, OpenVLA-OFT, and $\pi_{0.5}$, respectively. In contrast, TAPT with the VLA tool-family interface consistently improves over the corresponding standalone SFT baselines, with gains of up to $+5.2$ points on LIBERO-Long and $+35.5$ points on RoboTwin. These results indicate that the main benefit does not come from wrapping a VLA with a planner alone, but from making the VLA executor tool-aligned and reliable under agent–tool interaction.

Table 3: Core ablation on LIBERO-Long. We ablate TAPT and the VLA tool-family interface across OpenVLA-OFT and $\pi_{0.5}$, where TAPT includes IA SFT/Post-train and tool-family residual adapters.

IA SFT	IA Post-train	Components		Success Rate (%) $\uparrow$	
		Tool-family residual adapters	VLA tool-family interface	OpenVLA-OFT	$\pi_{0.5}$
✓				92.0	92.4
✓				93.0	93.4
✓	✓			94.2	91.2
✓	✓	✓		94.8	96.0
			✓	80.2	80.4
✓	✓	✓	✓	95.6	97.2

Table 4: Ablation of agent-level replanning on LIBERO-Long. We compare the VLA tool-family interface with direct VLM monitoring in terms of intervention behavior and average calls per task.

Model	with VLA tool-family interface			VLM directly monitor
	Intervene Freq.	Replan SR	Avg. Calls / Task	Avg. Calls / Task
OpenVLA	40.8%	47.1%	1.988	109.5
OpenVLA-OFT	12.2%	34.4%	0.304	58.2
$\pi_{0.5}$	11.4%	61.4%	0.228	57.2

Reinforcement learning in the target domain. We next evaluate whether *VLAs-as-Tools* remains effective when VLAs are improved through target-domain RL. Figure 3(a) compares Standard RL, IA RL, TAPT, and TAPT with the VLA tool-family interface on LIBERO-Long. Here, Standard RL denotes full-task RL that optimizes the original long-horizon task success without IA units. The full configuration outperforms Standard RL by +3.8 and +11.2 points for OpenVLA-OFT and $\pi_{0.5}$, respectively. The VLA tool-family interface further adds +2.6 and +2.0 points over TAPT alone, showing the benefit of high-level planning and agent-tool interaction. Figure 3(b) evaluates CALVIN under its native subtask structure, where success also increases from 76.0 to 78.8 with Invocation-aligned RL and further to 82.3 with full TAPT. These results show that the proposed strategy also benefits RL-based training, and that the gains are not tied to our own split construction.

5.3 Does TAPT Make VLA Execution More Faithful to Agent Invocations?

The above results show that the VLA tool-family interface improves long-horizon task success, but success rate alone does not show whether the executor has become a better tool for an external agent. We therefore evaluate invocation fidelity on LIBERO-CF-Long, a counterfactual suite derived from LIBERO-10 scenes and inspired by LIBERO-CF [Fang et al., 2026]. Each task changes the requested goal relative to a familiar source task, for example by replacing target objects, changing spatial relations, or truncating a multi-step instruction. During evaluation, the high-level plans are fixed; we vary only the VLA executor by progressively enabling TAPT components.³

We use two invocation-level metrics to distinguish correct tool-call execution from source-task bias. Faithful Rate measures how often the rollout completes the counterfactual goal specified by the invocation. Biased Rate measures how often the rollout also proceeds toward the original LIBERO-10 source-task goal, reflecting over-execution or fallback to the training-task sequence. We report Non-biased Rate as $100\% - \text{Biased Rate}$, where higher is better.

Table 2 shows that TAPT improves the interface property that the agent actually needs: the executor becomes more likely to follow the requested invocation while becoming less likely to continue into the source-task bias. From SFT to full TAPT, OpenVLA-OFT improves by +34.6 points in Faithful Rate and +16.2 points in Non-biased Rate, while $\pi_{0.5}$ improves by +30.0 and +15.0 points, respectively. These gains indicate that TAPT does not merely increase task success, but strengthens the invocation-behavior binding required for reliable tool use.

5.4 Core Design Ablation

Table 3 answers three questions about the contributions of VLAs-as-Tools components. First, subtask-centric supervision with invocation-aligned SFT improves both model type by 1%. Second, TAPT further improves OpenVLA-OFT steadily to 94.8%, but $\pi_{0.5}$ shows a backbone-specific trade-off: IA Post-train alone reduces success from 93.4% to 91.2%, likely because the available open-source data is less matched to the physical-intelligence high quality training distribution. Tool-family residual adapters recover this drop and improve success to 96.0%. Third, the interface-only setting performs poorly, showing that planner wrapping alone cannot make a non-tool-aligned executor reliable. The

³The original LIBERO-CF benchmark is not publicly released. We evaluate on our own hand-designed LIBERO-CF-Long reconstruction, using familiar LIBERO-10 layouts with counterfactual goal edits and source-task biased-goal checks.

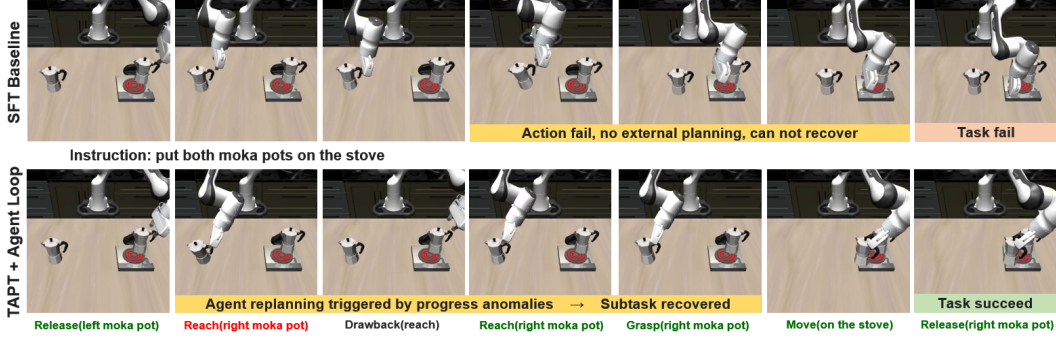


Figure 4: Qualitative comparison of long-horizon failure recovery. Our method prevents cascading failures through progress feedback and replanning, turning local errors into recoverable transitions.

full configuration achieves the best results, 95.6% for OpenVLA-OFT and 97.2% for $\pi_{0.5}$, confirming that TAPT and the VLA tool-family interface jointly enable effective tool use.

5.5 Agent planning efficiency

Table 4 compares progress-based replanning with a direct VLM monitoring baseline that queries the VLM every 5 steps. The VLA tool-family interface substantially reduces high-level calls, from 109.5, 58.2, and 57.2 per episode to only 1.988, 0.304, and 0.228, respectively. Despite the much lower query cost, progress-triggered intervention still achieves replan success rates of 47.1%, 34.4%, and 61.4%, showing that progress feedback offers a lightweight alternative to frequent VLM monitoring.

5.6 Tool family residual adapter cost

Table 5 compares the normalized cost of adding tool-family residual adapters. The adapters add a small overhead to OpenVLA-OFT, increasing parameters by 9% and inference time by 7%. On the lighter $\pi_{0.5}$ backbone, the adapted model remains only $0.50\times$ the OpenVLA/OpenVLA-OFT parameter baseline, although its inference time is higher. These results indicate that the tool-family parameterization provides specialization without requiring a separate full VLA model for each tool family.

Table 5: Normalized model-size and inference-time on LIBERO-Long. Values are normalized to OpenVLA-OFT without tool families.

Model	Parameters	Inference time
OpenVLA	1.00×	1.00×
OpenVLA-OFT + tool families	1.09×	1.07×
$\pi_{0.5}$	0.44×	1.26×
$\pi_{0.5}$ + tool families	0.50×	1.34×

5.7 Qualitative Analysis

Figure 4 qualitatively illustrates why treating the VLA as a callable tool improves long-horizon robustness. In the baseline rollout without replanning, the robot drops the left moka pot during execution and continues the remaining trajectory from an invalid intermediate state, causing the full task to fail. This behavior reflects a common failure mode of monolithic long-horizon policies: once an early manipulation error changes the scene state, later actions remain conditioned on the nominal trajectory rather than on a recovered subtask state. In contrast, our agent-tool loop monitors subtask progress, detects that the attempted grasp has failed, withdraws from the failed interaction, and re-invokes the corresponding VLA tool before continuing to the next pot. The successful rollout therefore decomposes recovery into a bounded local correction rather than requiring the VLA executor to solve the entire long-horizon task in one uninterrupted policy rollout.

6 Conclusion

We presented **VLA-as-Tools**, a framework that treats vision-language-action models as callable, specialized executors under a planning-capable multimodal agent. By separating high-level decomposition, progress monitoring, and recovery from bounded robot-control execution, the framework makes VLA policies more composable for long-horizon embodied tasks. We further introduced Tool-Aligned Post-Training to improve invocation fidelity and specialize VLA tools without discarding their pretrained semantic capabilities. Experiments on LIBERO-Long show that this agent-tool formulation substantially improves task success, instruction alignment, few-shot adaptation, and robustness across multiple planner backbones. A current limitation is that our study focuses on simulated manipulation benchmarks; extending the same tool interface and post-training recipe to diverse real-robot settings is an important direction for future work.

References

- Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- Suneel Belkhale, Tianli Ding, Ted Xiao, et al. RT-H: Action hierarchies using language. *arXiv preprint arXiv:2403.01823*, 2024.
- Kevin Black, Noah Brown, Danny Driess, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023a.
- Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023b.
- Kaiyuan Chen, Shuangyu Xie, Zehan Ma, Pannag R Sanketi, and Ken Goldberg. Robo2vlm: Visual question answering from large-scale in-the-wild robot manipulation datasets, 2025. URL <https://arxiv.org/abs/2505.15517>.
- Yiguo Fan, Pengxiang Ding, Shuanghao Bai, Xinyang Tong, Yuyang Zhu, Hongchao Lu, Fengqi Dai, Wei Zhao, Yang Liu, Siteng Huang, Zhaoxin Fan, Badong Chen, and Donglin Wang. Long-VLA: Unleashing long-horizon capability of vision language action model for robot manipulation. *arXiv preprint arXiv:2508.19958*, 2025.
- Yu Fang, Yuchun Feng, Dong Jing, Jiaqi Liu, Yue Yang, Zhenyu Wei, Daniel Szafr, and Mingyu Ding. When vision overrides language: Evaluating and mitigating counterfactual failures in VLAs. *arXiv preprint arXiv:2602.17659*, 2026.
- Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. Cast: Counterfactual labels improve instruction following in vision-language-action models, 2025.
- Asher J. Hancock, Xindi Wu, Lihan Zha, Olga Russakovsky, and Anirudha Majumdar. Actions as language: Fine-tuning VLMs into VLAs without catastrophic forgetting. *arXiv preprint arXiv:2509.22195*, 2025.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. ThinkAct: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025.
- Eric Huang, Xianyi Cheng, Yuemin Mao, Arnav Gupta, and Matthew T Mason. Autogenerated manipulation primitives. *The International Journal of Robotics Research*, 42(6):433–458, 2023. doi: 10.1177/02783649231170897.
- Wenlong Huang, Fei Xia, Ted Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- Eric Jang, Alex Irpan, Mohi Khansari, et al. BC-Z: Zero-shot task generalization with robotic imitation learning. *arXiv preprint arXiv:2202.02005*, 2022.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. DROID: A large-scale in-the-wild robot manipulation dataset, 2024. URL <https://arxiv.org/abs/2403.12945>.

- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- Ruiying Li, Yunlang Zhou, Yuyao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, Jiayao Ma, Xin He, Yongjian Shen, Yang Yang, Guanghui Ren, Maoqing Yao, Wenhao Wang, and Yao Mu. RoboClaw: An agentic framework for scalable long-horizon robotic tasks, 2026.
- Jacky Liang, Wenlong Huang, Fei Xia, et al. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022.
- Wenlong Liang, Rui Zhou, Yang Ma, Bing Zhang, Songlin Li, Yijia Liao, and Ping Kuang. Large model empowered embodied ai: A survey on decision-making and embodied learning, 2025.
- Bo Liu, Yifeng Zhu, Chongkai Gao, et al. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.
- Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- Yuan Liu, Haoran Li, Shuai Tian, Yuxing Qin, Yuhui Chen, Yupeng Zheng, Yongzhen Huang, and Dongbin Zhao. Towards long-lived robots: Continual learning VLA models via reinforcement fine-tuning. *arXiv preprint arXiv:2602.10503*, 2026a.
- Yuanzhe Liu, Jingyuan Zhu, Yuchen Mo, Gen Li, Xu Cao, Jin Jin, Yifan Shen, Zhengyuan Li, Tianjiao Yu, Wenzhen Yuan, Fangqiang Ding, and Ismini Lourentzou. PALM: Progress-aware policy learning via affordance reasoning for long-horizon robotic manipulation. *arXiv preprint arXiv:2601.07060*, 2026b.
- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *arXiv preprint arXiv:2112.03227*, 2021.
- Oier Mees, Lukas Hermann, and Wolfram Burgard. What matters in language conditioned robotic imitation learning over unstructured data. *arXiv preprint arXiv:2204.06252*, 2022.
- Runqing Miao, Qingxuan Jia, Fuchun Sun, Gang Chen, Haiming Huang, and Shengyi Miao. Semantic representation of robot manipulation with knowledge graph. *Entropy*, 25(4):657, 2023. doi: 10.3390/e25040657.
- Yao Mu, Tianxing Chen, Zanzin Chen, et al. RoboTwin: Dual-arm robot benchmark with generative digital twins. *arXiv preprint arXiv:2504.13059*, 2025.
- NVIDIA, Johan Bjorck, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Octo Model Team, Dibya Ghosh, Homer Walke, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- Open X-Embodiment Collaboration, Abby O’Neill, Abdul Rehman, Abhinav Gupta, et al. Open X-Embodiment: Robotic learning datasets and RT-X models. *arXiv preprint arXiv:2310.08864*, 2023.
- Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Sahar Salimpour, Lei Fu, Kajetan Rachwał, Pascal Bertrand, Kevin O’Sullivan, Robert Jakob, Farhad Keramat, Leonardo Militano, Giovanni Toffetti, Harry Edelman, and Jorge Peña Queralta. Towards embodied agentic ai: Review and classification of llm- and vlm-driven robot autonomy and interaction, 2025.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, et al. Hi Robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.
- Zhe Su, Oliver Kroemer, Gerald E. Loeb, Gaurav S. Sukhatme, and Stefan Schaal. *Learning to Switch Between Sensorimotor Primitives Using Multimodal Haptic Signals*, pages 170–182. Springer International Publishing, 2016. doi: 10.1007/978-3-319-43488-9_16.
- Zhe Su, Oliver Kroemer, Gerald E. Loeb, Gaurav S. Sukhatme, and Stefan Schaal. Learning manipulation graphs from demonstrations using multimodal sensory signals. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2758–2765. IEEE, 2018. doi: 10.1109/ICRA.2018.8461121.
- Xiaoquan Sun, Zetian Xu, Chen Cao, Zonghe Liu, Yihan Sun, Jingrui Pang, Ruijian Zhang, Zhen Yang, Kang Pang, Dingxin He, Mingqi Yuan, and Jiayu Chen. AtomVLA: Scalable post-training for robotic manipulation via predictive latent world models. *arXiv preprint arXiv:2603.08519*, 2026.
- Homer Rich Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1723–1736. PMLR, 2023. URL <https://proceedings.mlr.press/v229/walke23a.html>.
- Junjie Wen, Yichen Zhu, Jinming Li, et al. DexVLA: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025.
- John Yang, Carlos E. Jimenez, Alexander Wettig, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- Yi Yang, Jiaxuan Sun, Siqi Kou, Yihan Wang, and Zhijie Deng. LoHoVLA: A unified vision-language-action model for long-horizon embodied tasks. *arXiv preprint arXiv:2506.00411*, 2025a.
- Yue Yang, Shuo Cheng, Yu Fang, Homanga Bharadhwaj, Mingyu Ding, Gedas Bertasius, and Daniel Szafir. LiLo-VLA: Compositional long-horizon manipulation via linked object-centric policies. *arXiv preprint arXiv:2602.21531*, 2026.
- Zhejian Yang, Yongchao Chen, Xueyang Zhou, Jiangyue Yan, Dingjie Song, Yinuo Liu, Yuting Li, Yu Zhang, Pan Zhou, Hechang Chen, and Lichao Sun. Agentic robot: A brain-inspired framework for vision-language-action models in embodied agents. *arXiv preprint arXiv:2505.23450*, 2025b.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Philipp Zech, Erwan Renaudo, Simon Haller, Xiang Zhang, and Justus Piater. Action representations in robotics: A taxonomy and systematic classification. *The International Journal of Robotics Research*, 38(5):518–562, 2019. doi: 10.1177/0278364919835020.
- Likui Zhang, Tao Tang, Zhihao Zhan, Xiuwei Chen, Zisheng Chen, Jianhua Han, Jiangtong Zhu, Pei Xu, Hang Xu, Hefeng Wu, Liang Lin, and Xiaodan Liang. AtomicVLA: Unlocking the potential of atomic skill learning in robots. *arXiv preprint arXiv:2603.07648*, 2026a.
- Ninghao Zhang, Bin Zhu, Shijie Zhou, and Jingjing Chen. Restoring linguistic grounding in VLA models via train-free attention recalibration. *arXiv preprint arXiv:2603.06001*, 2026b.

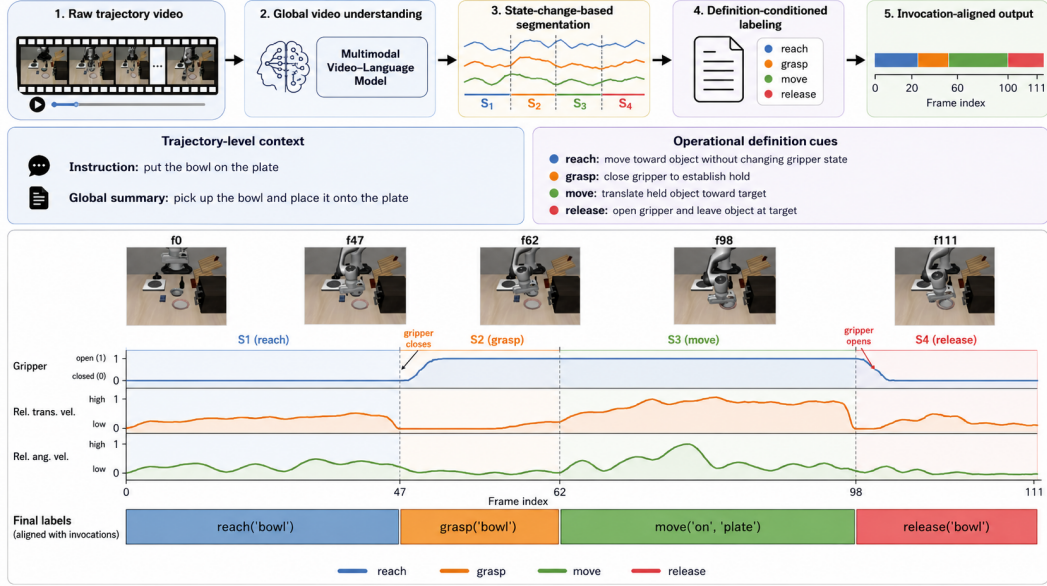


Figure 5: Visualization of the automatic labeling pipeline output. Raw robot trajectories are segmented into bounded manipulation clips and assigned invocation-aligned tool-family labels with normalized progress annotations, providing structured examples for Tool-Aligned Post-Training and downstream split adaptation.

A Automatic Labeling Pipeline

This appendix describes the automatic pipeline used to convert raw robot videos into invocation-aligned tool-family labels. The goal of the pipeline is not to infer an unconstrained natural-language summary of a trajectory, but to assign each bounded manipulation segment a discrete tool-family label whose execution semantics match the VLA invocation interface. We therefore combine global video understanding, state-change-based temporal segmentation, and definition-conditioned multimodal labeling. This design follows recent efforts in large-scale language-conditioned robot datasets and VLA learning Walke et al. [2023], Khazatsky et al. [2024], Open X-Embodiment Collaboration et al. [2023], Brohan et al. [2023a], but differs by producing invocation-aligned tool-family labels rather than trajectory-level task descriptions. We use gripper and motion-derived state changes as temporal cues, following prior work that segments manipulation demonstrations using proprioceptive, force, haptic, or contact-state signals Su et al. [2016, 2018], Chen et al. [2025]. The label space is grounded in manipulation primitive and robotic action representations Zech et al. [2019], Miao et al. [2023], Huang et al. [2023], with definitions expressed in terms of contact, force direction, object motion, and termination conditions. Figure 5 visualizes the overall output of the automatic labeling pipeline, including the segmented manipulation clips, their corresponding tool-family labels, and the aligned progress annotations.

Global video understanding. For each full trajectory, a multimodal large model first reads the complete video and produces a compact global semantic description. This pass identifies the manipulated objects, likely task intent, relevant spatial relations, and major contact events. The global summary is retained as trajectory-level context for later decisions, so that local segments can be interpreted with knowledge of the full task rather than only the visual evidence inside a short clip.

State-change-based segmentation. We then split the trajectory into candidate action segments using changes in the robot gripper state. Gripper opening and closing are treated as *explicit state-change signals* because they mark discrete interaction events and directly support labels such as grasp and release when such labels are used by a benchmark-specific taxonomy. In addition, we compute *reference state-change signals* from the relative translational velocity and relative angular velocity between the end-effector and the manipulated object. These reference signals are not used as hard labels by themselves. Instead, they provide temporal evidence for identifying non-gripper tool

Table 6: Operational definitions used by the automatic labeling pipeline. Arguments indicate the manipulated object, target object or target position, and geometric parameter when applicable.

Tool family	Definition
<code>reach(obj)</code>	The arm moves toward the object while the gripper state remains unchanged. The segment ends before grasping, contact-driven translation, or another object-changing interaction begins.
<code>move(pos, obj2)</code>	The arm translates toward a target position while the gripper state remains unchanged. Here <code>obj2</code> denotes the reference object or target area and <code>pos</code> denotes the target position relative to it.
<code>flip(obj, angle)</code>	The robot rotates the object, typically around a horizontal axis, so that its vertical orientation is inverted or the opposite side is exposed.
<code>rotate(obj, angle)</code>	The robot adjusts the object’s 3D pose or orientation without vertical inversion, or manipulates an articulated mechanism through a rotational joint.
<code>push(obj, pos)</code>	The end-effector contacts the object and translates while applying forward force, causing the object to move toward the target position. The action ends when object motion stops and contact is released.
<code>pull(obj, pos)</code>	The end-effector establishes a pulling contact and translates backward, causing the object to move toward the target position.
<code>insert(obj1, obj2)</code>	<code>obj1</code> is aligned with an opening, slot, or receptacle of <code>obj2</code> , then translated along the insertion axis until it is partially or fully seated.
<code>press(obj)</code>	The end-effector moves downward to contact the object and applies sustained normal force, usually with limited tangential motion.
<code>strike(obj1, obj2, pos)</code>	<code>obj1</code> , the end-effector, or another held item is moved rapidly to impact <code>obj2</code> or a specified location <code>pos</code> .

families such as push, pull, flip, rotate, insert, press, and strike. The resulting candidate windows are short enough for fine-grained action-boundary recognition while still preserving the local physical effect of the action.

Definition-conditioned tool-family labeling. For each candidate segment, the multimodal model receives four inputs: the segment video, the full-trajectory semantic summary, the explicit and reference state-change signals, and the written definitions of the tool families. It then assigns the segment to the tool family whose definition best explains the observed contact, motion direction, object displacement, and termination condition. The definitions are intentionally operational rather than purely linguistic: they specify when an action starts, what physical effect should occur, and when the action ends. For example, a push segment begins when the end-effector approaches the object and comes to a controlled contact, continues while the robot translates the object along a relatively fixed direction under forward force, and ends when the object stops moving and the end-effector separates from it. This definition prevents short approach motions, incidental contacts, and post-contact retreats from being mislabeled as pushes.

Signal use and consistency checks. The labeling model treats gripper open–close transitions as decisive evidence for interaction boundaries, but uses relative velocity and angular velocity as supporting evidence for the tool-family decision. High relative translational velocity with sustained contact supports push or pull depending on the force direction; high relative angular velocity supports flip or rotate depending on whether the object’s vertical orientation is inverted; a dominant translation along an opening axis supports insert; downward motion followed by sustained contact supports press; and a short high-speed impact supports strike. When the local visual evidence conflicts with the global trajectory context or with the operational definition, the segment is either assigned to the closest valid family with a lower confidence score or marked for manual review. This conservative rule keeps the automatically labeled corpus aligned with the intended invocation semantics rather than forcing ambiguous clips into an ill-matched family.

The final output of the pipeline is a set of invocation-aligned examples $(o_{1:T}, l, c, a_{1:T}, p)$, where c is the assigned tool-family label and p is a normalized progress signal within the segment.

Table 7: Completion predicates used to instantiate $\psi_{z,g}$ in RLinf. We reuse LIBERO native predicates for benchmark-defined spatial relations and add state-based predicates for intermediate tool invocations.

Invocation family g	Predicate source	Success condition for $\psi_{z,g}$
Reach	Added state predicate	The end-effector is within a task-specific distance threshold of the target object, handle, drawer, or knob.
Grasp	Added state predicate	The target object is held by the gripper, verified using contact and gripper-state information.
Move	LIBERO native predicate	The target object satisfies LIBERO’s native containment predicate with the target receptacle or on-top predicate with the target surface.
Release	Added state predicate	The gripper is open and the object is no longer held; for placement subtasks, the required spatial relation must remain satisfied.
Rotate	Added state predicate	The relevant articulated state, such as a stove-knob angle, crosses the task-specific threshold.
Push	Added state predicate	The object is displaced into the target region or satisfies the corresponding scene-state condition.

B Additional Details for Tool-Aligned Post-Training

This appendix records the formal details that are omitted from the main Method section for readability. For reinforcement learning, each invocation (z, g) defines a bounded subtask MDP whose reset distribution starts from states where that invocation should begin. Let $\mathcal{S}_{z,g}^0$ be the pool of such states, obtained from demonstration boundaries or successful executions of preceding subtasks. We sample

$$\rho_{z,g}(s) = \frac{1}{|\mathcal{S}_{z,g}^0|} \sum_{\bar{s} \in \mathcal{S}_{z,g}^0} \delta(s = \bar{s}), \quad s_0 \sim \rho_{z,g}. \quad (7)$$

Each invocation has a state-based completion predicate $\psi_{z,g}(s) \in \{0, 1\}$. For a bounded rollout $\tau = (s_0, a_0, \dots, s_H)$, the invocation-level reward is

$$R(\tau; z, g) = \mathbf{1} \left[\max_{0 \leq t \leq H} \psi_{z,g}(s_t) = 1 \right]. \quad (8)$$

Thus the RL stage optimizes the expected completion of the current invocation,

$$\max_{\theta, \Phi} \mathbb{E}_{(z,g), \tau \sim \pi_{\theta, \Phi}(\cdot|z), \mathcal{M}_{z,g}} [R(\tau; z, g)], \quad (9)$$

which we instantiate with GRPO in our experiments.

Completion predicates in RLinf. We instantiate $\psi_{z,g}$ using simulator-state predicates. For spatial relations that are already part of the LIBERO benchmark definition, such as placing an object in a receptacle or on a target surface, we reuse LIBERO’s native success predicates. For intermediate invocations that are not exposed as standalone LIBERO task goals, such as reaching or grasping, we add lightweight predicates over robot and object state. This keeps the final task relations aligned with LIBERO while still allowing RL to optimize bounded intermediate tool calls.

Reset-state pool refresh. The choice of $\mathcal{S}_{z,g}^0$ is important for composing independently trained subtask policies. In the first round of subtask RL, we collect reset states using the initial policy or demonstration boundaries. This provides valid states for training individual subtasks, but it can become stale after policy improvement: once a preceding subtask policy is updated, it may end with different gripper poses, object contacts, approach directions, or small object displacements than those in the original pool. Consequently, a downstream subtask may be trained on a reset distribution that no longer matches the states produced by the learned composed policy. In our experiments, this mismatch can make individually improved subtasks compose worse, sometimes reducing full-task success.

Table 8: Reset-state pool choices in RLinf. Refreshing $\mathcal{S}_{z,g}^0$ with the trained MoE policy reduces the distribution shift introduced when subtask RL changes the states passed between invocations.

Reset-state pool $\mathcal{S}_{z,g}^0$	What it matches	Observed effect
Original task resets	The benchmark initial state distribution	Appropriate for the first invocation, but unsuitable for later subtasks because the policy must implicitly reproduce the task prefix.
Initial-policy or demonstration boundary pool	States where each subtask should begin before subtask RL changes the policy	Enables first-round subtask RL, but can become mismatched after upstream subtasks are updated.
Refreshed MoE boundary pool	Boundary states produced by the trained composed subtask policy	Improves subtask handoff and increases full-task success by aligning training resets with deployment states.

Table 9: Few-shot downstream adaptation on LIBERO-Long. Results report success rate under limited SFT demonstrations.

	Base model	Method	10-shot	20-shot
	OpenVLA-OFT	Baseline	8.0	33.8
		VLAs-as-Tools	19.5	36.4
$\pi_{0.5}$		Baseline	43.4	60.0
		VLAs-as-Tools	49.8	65.6

To reduce this distribution shift, we refresh the reset-state pools after one round of subtask RL. We deploy the trained mixture-of-experts subtask policy, save boundary states whenever the preceding invocation succeeds, and use these refreshed states as $\mathcal{S}_{z,g}^0$ for the next round of subtask training. The refreshed pool therefore matches the state distribution induced by the current composed policy, improving the handoff between neighboring invocations. Empirically, this refresh step recovers full-task performance gains that are not obtained from stale initial-policy pools alone.

For the tool-family residual parameterization, each selected linear layer $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ receives a family-specific LoRA residual $\Delta W_g = B_g A_g$, where $A_g \in \mathbb{R}^{r \times d_{\text{in}}}$, $B_g \in \mathbb{R}^{d_{\text{out}} \times r}$, and $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Given hidden activation x , the adapted layer computes $y = Wx + \Delta W_g x$. For selected layers $\mathcal{L}$ and $K = |\mathcal{G}|$ tool families, this adds $\mathcal{O}(|\mathcal{L}|Kr(d_{\text{in}} + d_{\text{out}}))$ parameters while keeping the pretrained VLA backbone shared across families.

C Additional Adaptation and Planner Robustness Results

A central purpose of Tool-Aligned Post-Training is to make the VLA interface easier to adapt to a downstream benchmark. We therefore evaluate few-shot invocation-aligned SFT on LIBERO-Long under limited downstream demonstrations. If the invocation-aligned post-training stage has established reusable invocation semantics, the model should require substantially fewer downstream demonstrations to recover strong downstream performance. Table 9 shows that TAPT improves downstream sample efficiency, with the largest gain in the most data-limited OpenVLA-OFT 10-shot setting (+11.5 points) and consistent gains for $\pi_{0.5}$ under both 10-shot and 20-shot adaptation (+6.4 and +5.6 points).

We additionally test whether the agent–tool interface depends on a particular large VLM planner. In this experiment, the OpenVLA-OFT VLA tool families are fixed and only the high-level VLM is changed. Table 10 shows that the interface is not tied to a single high-level planner: across four strong VLM planners, LIBERO-Long success varies by only 0.6 points.

Table 10: High-level VLM planner ablation on LIBERO-Long under the OpenVLA-OFT VLA tool families.

Model type	Large VLM	Success Rate $\uparrow$
Close-Sourced	Gemini 3 Flash	95.6
	GPT 5.4	95.4
Open-Sourced	Qwen3.5-397B-A17B	95.4
	Qwen3VL-235B-A22B	95.0

D Evaluation Protocol Details

This appendix summarizes the evaluation protocol used for the task-level success rates reported in the main experiments. Unless otherwise specified, all methods compared within the same benchmark use the same held-out tasks, initial-state files, rollout horizons, random seeds, and success predicates. Success rate is computed as the fraction of evaluation episodes that satisfy the benchmark-defined task predicate before timeout. We follow each simulator’s standard evaluation setting and run 50 trials for each evaluated task unless otherwise specified.

LIBERO-Long. For task-level success evaluation, each policy is rolled out from the held-out LIBERO-Long initial states and is counted as successful if the native LIBERO task predicate is satisfied within the simulator’s standard rollout horizon. The same task split and initial-state set are used for standalone VLA policies, direct-planner deployment, and VLA tool-family interface deployment.

RoboTwin. Because our demonstration-splitting pipeline targets single-arm manipulation, we evaluate RoboTwin on the subset of 8 tasks that can be executed with a single Franka arm. A task is included if its successful demonstrations do not require bimanual coordination and can be segmented into the tool-family invocation types used in our interface. The selected tasks are:

adjust_bottle, click_alarmclock, move_stapler_pad, press_stapler,
beat_block_hammer, click_bell, place_empty_cup, place_shoe.

RoboTwin success is measured by the benchmark task predicate at the end of each rollout or when the task terminates successfully.

CALVIN. For CALVIN, we use CALVIN_D and split episodes into 80% training and 20% testing. The evaluation follows the benchmark’s native subtask structure: a rollout is successful if it completes the required long-horizon instruction sequence under CALVIN’s state-based success checks. In the RL experiments, the native subtask annotations are also used to define invocation-aligned resets and rewards, which lets us evaluate the proposed strategy without relying on our own split construction.

LIBERO-CF-Long. LIBERO-CF-Long evaluates invocation fidelity rather than only task success. We construct 10 counterfactual tasks from LIBERO-10 layouts by modifying the requested goal while keeping the scene visually close to a familiar source task. We report Faithful Rate, the fraction of rollouts satisfying the active counterfactual goal, and Non-biased Rate, defined as $100 - \text{Biased Rate}$, where Biased Rate measures whether the rollout also satisfies the paired source LIBERO-10 goal during a post-success continuation window. The continuation window length is 100.